

Informe proyecto 3

Nicolás Sanchez 202420978

Emanuela Martinez 202420655

Grupo 13

Repositorio: https://github.com/NicoSanchPort10/proyecto2_LYM.git

Introducción

Este documento describe los errores encontrados en la gramática e implementación del Proyecto 1, los cambios realizados para corregirlos en el Proyecto 2, y las nuevas funcionalidades implementadas en la Parte B del proyecto.

Errores del Proyecto 1 y cambios realizados

En la retroalimentación nos dijeron que la definición del lenguaje no podía contener paréntesis ni corchetes para definir la cantidad de atributos que podían tener como si fuera un regex. Es por esto que para la implementación del lenguaje en Rascal lo cambiamos. Por otra parte, eliminamos unos símbolos de puntuación que no eran necesarios. Y finalmente, redujimos significativamente la cantidad de no-terminales que había en la entrega del Proyecto 1.

1. Adición de charLiteral

Se agregó la regla léxica para literales de carácter, que estaba nombrada en la gramática pero sin definición concreta:

```
charLiteral ::= "" char ""  
            | "" \" char ""
```

Parte B — Nuevas funcionalidades implementadas

Para la segunda parte del proyecto se implementaron tres tareas principales que extienden el pipeline de VeriLang:

B1 — Integración del type checker en Run.rsc

Se extendió Run.rsc para que el reporte del pipeline incluya los resultados del type checker. Se agregó una función typeCheckReport(loc source) que parsea el archivo fuente, invoca checkProgram/1 del módulo Verilang::Checker, obtiene los mensajes con getMessages(tm) y los formatea como strings. El output ahora muestra una sección "Type checking" entre el resultado del round-trip y el código generado, indicando "OK — no errors" cuando no hay errores, o listando cada error con su línea. Además, runCli retorna código de salida 1 si el round-trip falla o si hay errores de tipo.

B2 — Formato de variables en Generate.rsc

Se modificó generateDecl(varDecl(...)) para que cada variable se muestre en una línea separada en lugar de usar intercalate con comas. El output anterior era " Vars: x:Nat, y:Nat" y el nuevo formato es una variable por línea con padding consistente, mejorando la legibilidad del output generado.

B3 — Corrección de imports en Checker.rsc y CheckVerilang.rsc

Se corrigió un problema con la declaración `importDecl` en `Checker.rsc` que causaba un fallo en runtime por `TypePal`. También se actualizó `CheckVerilang.rsc` para usar rutas absolutas en los archivos de test, lo cual era necesario para que los tests del checker funcionaran correctamente desde la raíz del proyecto.

Resultados de las pruebas

Los tres comandos de verificación del proyecto pasan correctamente:

- `CompileVerilang examples/set.veri` — muestra el reporte completo con type checking y código generado.
- `VerifyVerilang` — 4/4 round-trip tests pasan: `set.veri`, `set-normalized.veri`, `exists.veri`, `literals.veri`.
- `CheckVerilang` — 4/4 checker tests pasan (0 errores / 2 errores / 1 error / 0 errores según el caso). Results: 4 passed, 0 failed.

Uso de la IA

Para facilitar la lectura y pruebas del evaluador, le pedimos a Claude que nos ayudara con la redacción del Readme dentro del repositorio, incluyendo las diferentes pruebas que puede realizar desde la raíz del proyecto y desde la terminal de Rascal.